

Economic Intelligence & Financial Markets Platform

Author: Diego Leonardo Alves de Andrade · **Live Dashboard:** Local deployment via Docker · **Source Code:** github.com/DiegoTDDD/Economic-data-platform

Executive Summary

Most data portfolio projects optimize a single metric in a static, localized notebook and stop there. I wanted to demonstrate the engineering that actually matters in industry: how data is extracted, stored, orchestrated, and served in a reproducible, containerized environment. I built an end-to-end ELT (Extract, Load, Transform) platform that ingests macroeconomic indicators (the IPCA inflation index and Unemployment Rate from the Central Bank of Brazil, plus the USD/BRL exchange rate) alongside cryptocurrency market data (Bitcoin), structures it in a strict PostgreSQL "Gold" layer, and serves it through interactive Streamlit dashboards.

The core value of this project is the robust infrastructure surrounding the data. I designed modular extraction scripts, a relational database schema optimized for time-series queries, Dockerized network isolation with dependency-aware startup ordering, and a continuous integration (CI/CD) pipeline. The result is a platform that extracts raw data from disparate APIs and promotes it to a business-ready state, the way a production system operates.

The Business Problem I Set Out to Solve

Financial analysts often need to correlate macroeconomic trends with volatile assets like Bitcoin to understand market cycles. The answers are publicly available, but they are buried across multiple APIs, returned in inconsistent JSON structures, and formatted with mismatched temporal granularities (daily crypto prices vs. monthly inflation rates). I wanted to turn that fragmented, messy data into a structured, queryable asset.

Guiding question: Can I build a fully isolated, containerized platform that reliably ingests disparate financial APIs, normalizes the data into a strict relational schema, and serves it visually — while backing the entire system with automated CI/CD checks?

I chose this angle deliberately. It forces the pipeline to reconcile schema differences, handle inconsistent API data types, enforce data quality before it reaches the dashboard, and maintain a historical record of economic indicators that a business can trust.

How I Designed the System

I treated this as a platform architecture problem, not a one-off script. The architecture separates concerns into distinct layers, so the same foundation can ingest these metrics today and scale to new data sources tomorrow.

1. Ingestion Layer (Python / APIs)

The ingestion layer consists of independent scripts (`bitcoin_ingestion.py`, `macro_ingestion.py`) responsible for fetching raw data. Instead of a monolithic script, I decoupled the extraction logic by domain. Each script handles its own API requests, parses the incoming JSON, and casts the data into pandas DataFrames — standardizing timestamps and enforcing types before any insertion is attempted, which prevents downstream schema corruption.

```
PS C:\Users\Usuario\projeto_economia_dados\infrastructure> docker compose logs pipeline
economics_pipeline | [*] Initializing Gold layer schema and analytical tables...
economics_pipeline | [*] Gold layer tables successfully initialized.
economics_pipeline | 2026-09-08 18:38:12,486 - INFO - Starting complete data platform orchestration...
economics_pipeline | 2026-09-08 18:38:12,486 - INFO - [Database Initialization] Running script: database_init.py
economics_pipeline | 2026-09-08 18:38:12,822 - INFO - Success in [Database Initialization]:
economics_pipeline | [*] Initializing Gold layer schema and analytical tables...
economics_pipeline | [*] Gold layer tables successfully initialized.
economics_pipeline | 2026-09-08 18:38:12,822 - INFO - [Bitcoin Data Ingestion] Running script: analytics_models/bitcoin_ingestion.py
economics_pipeline | 2026-09-08 18:38:16,889 - INFO - Success in [Bitcoin Data Ingestion]:
economics_pipeline | [*] Downloading historical Bitcoin (BTC-USD) data...
economics_pipeline | [*] Inserting 2443 records into the gold_bitcoin_metrics table...
economics_pipeline | [*] Bitcoin ingestion completed successfully.
economics_pipeline | 2026-09-08 18:38:16,889 - INFO - [Macroeconomic Data Ingestion] Running script: analytics_models/macro_ingestion
.py
economics_pipeline | 2026-09-08 18:38:23,839 - INFO - Success in [Macroeconomic Data Ingestion]:
economics_pipeline | [*] Fetching IPCA from Central Bank of Brazil (BCB SGS Series 433)...
economics_pipeline | [*] Successfully loaded 139 records for IPCA.
economics_pipeline | [*] Fetching Unemployment Rate from Central Bank of Brazil (BCB SGS Series 24369)...
economics_pipeline | [*] Successfully loaded 139 records for Unemployment Rate.
economics_pipeline | [*] Fetching USD/BRL Exchange Rate via yfinance...
economics_pipeline | [*] Successfully loaded 3040 records for USD/BRL Exchange Rate.
economics_pipeline | [*] Inserting total of 3318 macroeconomic records into gold_economic_indicators...
economics_pipeline | [*] Macroeconomic multi-indicator ingestion completed successfully with 0 errors.
economics_pipeline | 2026-09-08 18:38:23,839 - INFO - Pipeline executed successfully from end to end.
PS C:\Users\Usuario\projeto_economia_dados\infrastructure>
```

Live orchestration log — schema init, Bitcoin ingestion, and macro ingestion running end to end

2. Storage Layer (PostgreSQL Gold)

Rather than dumping raw JSON into a data lake, I chose to enforce schema-on-write. The data lands in a cleanly normalized PostgreSQL database. Macroeconomic time-series data benefits immensely from ACID guarantees, strict typing, and native SQL analytical functions. By structuring this as a "Gold" layer, downstream analytics become a predictable SQL query rather than a data-wrangling exercise.

```
economics_gold=# SELECT date, indicator_name, value FROM gold_economic_indicators
LIMIT 10;

```

date	indicator_name	value
2015-01-01	IPCA - Monthly (%)	1.24
2015-02-01	IPCA - Monthly (%)	1.22
2015-03-01	IPCA - Monthly (%)	1.32
2015-04-01	IPCA - Monthly (%)	0.71
2015-05-01	IPCA - Monthly (%)	0.74
2015-06-01	IPCA - Monthly (%)	0.79
2015-07-01	IPCA - Monthly (%)	0.62
2015-08-01	IPCA - Monthly (%)	0.22
2015-09-01	IPCA - Monthly (%)	0.54
2015-10-01	IPCA - Monthly (%)	0.82

(10 rows)

Direct SQL query against the Gold layer — data lands typed and query-ready

3. Orchestration & Containerization

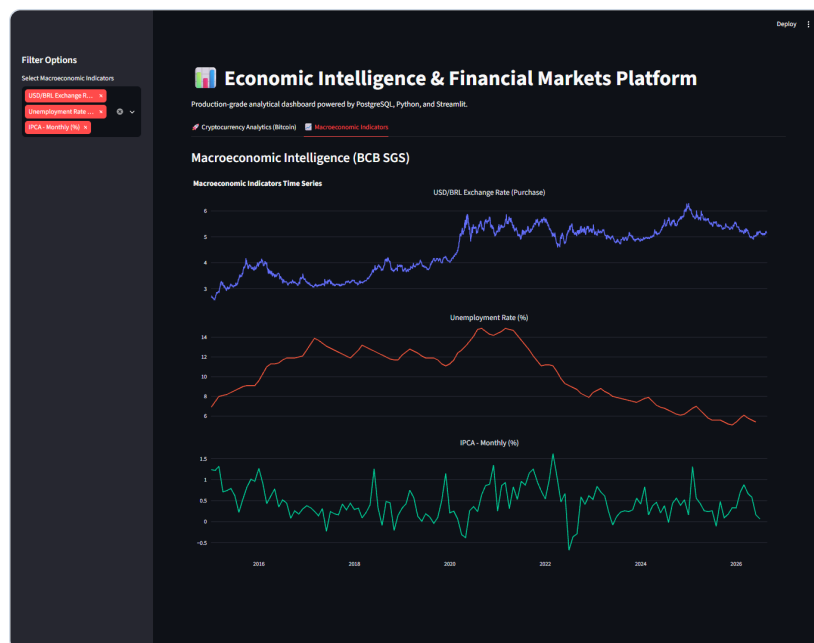
A master `orchestrator.py` script coordinates execution order — schema initialization, then Bitcoin ingestion, then macroeconomic ingestion — failing fast and logging clearly if any step errors out. The entire stack (database, pipeline, dashboard) is orchestrated via Docker Compose with an explicit health check on Postgres, so the application containers never attempt a connection before the database has actually finished initializing. This makes the system entirely reproducible on any machine, with no local dependency conflicts.

The Decision I Am Most Upfront About

I deliberately chose a traditional PostgreSQL relational database for the Gold layer rather than a distributed data lake (like Delta Lake or Apache Iceberg). For macroeconomic time-series data at this specific volume, the ACID guarantees, strict schema enforcement, and native SQL capabilities of Postgres are the correct engineering choice. Reaching for a massive distributed engine here would have been architecture theater. I'd rather demonstrate the judgment to pick the right tool for the scale than implement a buzzword that adds unnecessary overhead and maintenance burden.

Delivering the Result: Live Operations Console

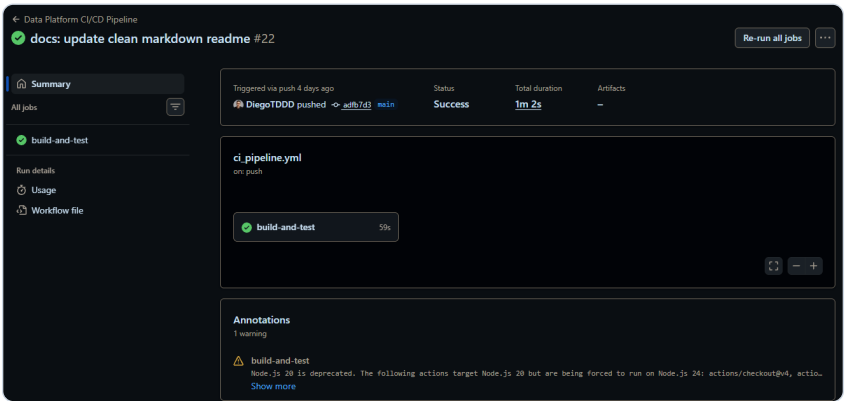
I published the analytical layer as an interactive Streamlit dashboard. Because the heavy lifting (filtering, sorting, aggregation) is handled by PostgreSQL, the dashboard simply executes optimized SQL queries and renders the results via Plotly — separating storage compute from presentation compute.



Macroeconomic dashboard — USD/BRL exchange rate, unemployment, IPCA

Ensuring the System Is Trustworthy (CI/CD)

A data platform is only as reliable as its testing infrastructure. I integrated a Continuous Integration pipeline using GitHub Actions. On every push, the pipeline spins up an isolated Ubuntu environment with a disposable PostgreSQL service, installs the exact pinned dependencies from `requirements.txt`, and runs the platform's test suite — so no faulty schema contract can be merged into `main` without immediately failing the build.



GitHub Actions — CI pipeline passing on the main branch

Real Engineering Friction, and How I Resolved It

Building this end-to-end surfaced real infrastructure challenges. I diagnosed each to root cause instead of papering over it — the unglamorous, decisive debugging that production work is made of.

ISSUE	ROOT CAUSE	FIX
Container Race Conditions	The ingestion pipeline attempted to connect to PostgreSQL before the database container had finished initializing and was ready to accept connections.	Added a Docker Compose health check (<code>pg_isready</code>) on the Postgres service and made the pipeline and dashboard containers declare <code>depends_on: condition: service_healthy</code> — so they never even start until the database confirms it's ready.
CI/CD Action Deprecation	GitHub Actions flagged "Node.js 20 is deprecated" during the build-and-test workflow, due to the underlying action versions in use.	Pinned the workflow to the latest available actions (<code>actions/checkout@v4</code> , <code>actions/setup-python@v5</code>). The residual Node.js runtime notice is a GitHub-hosted-runner platform notice, not something an individual workflow file can fully suppress — a useful distinction between what a fix controls and what it doesn't.

What This Project Demonstrates

- **Data Platform Design:** a modular ELT architecture separating extraction, storage, and serving concerns into distinct, maintainable layers.
- **Containerization:** a fully isolated Docker environment with dependency-aware startup, proving the project is reproducible on any machine and not tied to local environment state.
- **Relational Modeling:** strict schemas and SQL-based analytics in a PostgreSQL Gold layer to guarantee data integrity.
- **CI/CD Automation:** a GitHub Actions pipeline that gates code quality and automates testing on every push.
- **Honest Engineering:** prioritizing the right tool for the job over unnecessary complexity, and being precise about what a fix does and doesn't solve.

Technical Reference

STAGE	COMPONENT	PURPOSE
Ingestion	Python (requests, pandas)	API extraction, JSON parsing, and DataFrame normalization.
Storage (Gold Layer)	PostgreSQL	ACID-compliant relational storage for clean, queryable time-series data.
Orchestration	Python (orchestrator.py)	Execution sequencing and fail-fast error handling.
Infrastructure	Docker & Docker Compose	Isolated container network with health-check-based startup ordering.
CI/CD	GitHub Actions	Automated build-and-test workflow for continuous integration.
Presentation	Streamlit + Plotly	Interactive operations console and real-time visualization.

Data sources: Banco Central do Brasil — SGS API (IPCA inflation index and Unemployment Rate), and yfinance (USD/BRL exchange rate and Bitcoin price/volume). Raw data is dynamically extracted at runtime; no static datasets are hardcoded in the repository.